Revised Apr 30, 2024

Table of Contents

[1. Introduction 3](#introduction)

[1.1.1 Emulator vs Simulator 3](#emulator-vs-simulator)

[2. Whirlwind Assembler 5](#whirlwind-assembler)

[2.1 wwasm Command Line Options 5](#wwasm-command-line-options)

[2.2 Source Code Format 6](#source-code-format)

[2.3 Whirlwind operations – Op Codes and Operands 8](#whirlwind-operations-op-codes-and-operands)

[2.4 Numbers 8](#numbers)

[2.5 Expressions 9](#expressions)

[2.6 Labels 10](#labels)

[2.7 Pseudo-Ops 10](#pseudo-ops)

[2.7.1 .WW_TapeID: *string* 10](#ww_tapeid-string)

[2.7.2 .WW_File *string* 11](#ww_file-string)

[2.7.3 .isa [1950|1954] 11](#isa-19501954)

[2.7.4 .switch *name, integer* .switch FFRegAssign, *addr1*, . . . , *addrN* 11](#switch-name-integer-.switch-ffregassign-addr1-addrn)

[2.7.5 .dbwgt *addr*|*string* [, *step-value* [, *format-string*]] 12](#dbwgt-addrstring-step-value-format-string)

[2.7.6 .jumpto *addr* 12](#jumpto-addr)

[2.7.7 .org *addr* 12](#org-addr)

[2.7.8 .daorg *addr* 12](#daorg-addr)

[2.7.9 .pp *variable*, *expr* 13](#pp-variable-expr)

[2.7.10 .word *expr* 13](#word-expr)

[2.7.11 .words *size* [, *initial-value*] 13](#words-size-initial-value)

[2.7.12 .float *mantissa*:fraction, *exponent*:integer 13](#float-mantissafraction-exponentinteger)

[2.7.13 .flexh *string* [, *terminator*:integer] *.*flexl *string* [, *terminator*:integer] 14](#flexh-string-terminatorinteger-.flexl-string-terminatorinteger)

[2.7.14 .exec *python-stmt* 14](#exec-python-stmt)

[2.7.15 .print *format-str*, *addr1*, . . . ,*addrN* 15](#print-format-str-addr1-addrn)

[2.7.16 .include *filename* 15](#include-filename)

[*2.7.17* .if *<expr>* 15](#if-expr)

[2.7.18 .simparam 16](#simparam)

[2.8 Listing and Source File 18](#listing-and-source-file)

[2.9 CS-II Code Converter 19](#cs-ii-code-converter)

[2.10 Core File Format 19](#core-file-format)

[2.11 Assembler Grammar 20](#assembler-grammar)

[3. Simulator 23](#simulator)

[3.1 Command Line Options 23](#command-line-options)

[3.1.1 Program Launch Options 24](#program-launch-options)

[3.1.2 Display Options 26](#display-options)

[3.1.3 Instruction Trace 27](#instruction-trace)

[3.1.4 Specialized 28](#specialized)

[3.2 Control Panel 28](#control-panel)

[3.3 I/O Devices 30](#io-devices)

[3.3.1 Graphics Output and the Light Gun 30](#graphics-output-and-the-light-gun)

[3.4 Simulator Files 31](#simulator-files)

[3.4.1 Core Files 31](#core-files)

[3.4.2 Project_exec.py Files 31](#project_exec.py-files)

[3.5 Program Trace Log 31](#program-trace-log)

[3.6 Flow Graph 31](#flow-graph)

[4. Ancillary Tools 31](#ancillary-tools)

[4.1 wwdisasm – Disassembler for Whirlwind Binaries 31](#wwdisasm-disassembler-for-whirlwind-binaries)

[4.2 wwdiff – Compare core files 31](#wwdiff-compare-core-files)

[4.3 wwutd – Universal Tape Decoder 31](#wwutd-universal-tape-decoder)

[5. End of Document 31](#end-of-document)

Introduction

The Whirlwind computer was built at MIT, and operated there from about 1950 to 1959, during which time numerous documents were written, and much code produced, some of which has been preserved on paper and magnetic tapes.

As part of our work restoring software from this project at the MIT Museum and the Computer History Museum, we've developed a development environment to decode existing Whirlwind programs, or write new ones, and to run them in a simulation environment, called the MitWiSE, MIT Whirlwind Simulation Environment.

This document outlines how to use the components of the MitWiSE.

The Environment contains a number of components:

- wwsim - An instruction set simulator that can execute Whirlwind programs
- wwasm – An assembler that can translate a modern-format source code representation of the instruction set into binary opcodes.
- wwdisasm – a disassembler that decodes (to the extent possible) existing Whirlwind binary programs, producing a format that can be easily commented and labelled, then converted back into executable binary format with embedded labels by the assembler.
- csii-convert offers a translator to convert source files written in Comprehensive System format into the MitWiSE assembler format, again, to allow code to be inspected, commented, labeled and assembled to an executable.

All of the tools are written in Python (approximately Python 3.11), and most can be found on GitHub at:

https://github.com/gfedorkow/Whirlwind-Instruction-Simulator

All of the tools, excepting the simulator itself, are command-line oriented, running in a Linux or emulated-Linux environment like Cygwin on Windows.

Emulator vs Simulator

Names Matter (up to a point)... there are Simulators, there are Emulators. Which is this?

At the extremes, a Simulator produces a complete environment disjoint from the physical world that can produce behavior similar to some other system. For example, a climate simulator is clearly a model that we want to behave like the real climate, but no one is going to confuse one for the other.

An Emulator is an object which is designed to stand in for another object. At the extreme, Control Data (and many others) made devices that 'emulated', i.e., were plug-compatible with, IBM disk drives. With a faithful emulation, the

insides of the disk drive could be completely different from the originals, but they would operate interchangeably when plugged into an IBM mainframe.[1]

Our Whirlwind environment is half-way in between. It is a synthetic environment isolated from any dependence on specific physical object (i.e., a simulator), but it does run real Whirlwind code, without modification, at approximately real-time speed (i.e., an emulator).

But in the end, the difference doesn't matter much, so for now, it's a Simulator.

Whirlwind Assembler

The wwasm assembler is used to convert programs written in human-readable assembly language into code that can be executed in the MIT Whirlwind Simulator.

The input language inherits some characteristics of the original WW assembler (aka 'converter'), but adds on a number of more-modern constructs to allow for programs that, while (insignificantly) harder to parse, are substantially easier to document.[2]

wwasm Command Line Options

$ wwasm -h

usage: Assemble a Whirlwind Program. [-h] [--LogDir LOGDIR]

[--outputfilebase OUTPUTFILEBASE]

[--Verbose] [--Debug] [--MinimalListing]

[-D] [--ISA_1950]

inputfile

positional arguments:

inputfile file name of ww asm source file

options:

-h, --help show this help message and exit

--LogDir LOGDIR Directory into which to store logs. Default is current

wd.

--outputfilebase OUTPUTFILEBASE, -o OUTPUTFILEBASE

[1] Until they didn't, at which point a fraught debug session would ensue to assign 'blame' to one side of the plug or the other. Lawyers are standing by.

[2] i.e., variable names don't have to be one letter followed by up to three digits. And you can put in comments.

base name for output file

--Verbose, -v print progress messages [currently none]

--Debug, -d Print lotsa debug info [currently none]

--MinimalListing Do not include prefix address and auto-comments in listing

-D, --DecimalAddresses

Display listing addresses in decimal as well as octal

--ISA_1950 Use the 1950 version of the instruction set

Source Code Format

We'll start with a basic example, add.ww:

.org 0o40 Establishes program start at octal 40

.jumpto main Tells simulator to begin execution at main

main: ca x Load x into the AC

ad y Add y into the AC

ts z Store AC in z

.print "%d %d %d", x, y, z executable pseudo-op, prints results

si 0 Select the zero device, i.e. halt

x: .word 10 Allocate a word with label x and contents 10

y: .word 20

z: .word 0

Here is the result of a run:

$ wwasm add.ww

Listing output to file add.lst

Corefile output to file add.acore

wwsim -q add.acore

start at 0o40

x = 10, y = 20, z = 30

Halt Instruction! (Code=0) at pc=043

Alarm 'Program Halt' (5) at PC=0o43 (0d35)

Ran 3 cycles; Used mem=48MB

$

The syntax is formalized (to an extent) in the later section "Assembler Grammar."

A Whirlwind assembly program (.ww) consists of a sequence of text lines, each of which may be either a WW operation, a pseudo-op, a comment, or blank space.

The basic format of a line is:

@<addr>:<value> <label>: <instruction> ; <comment> @@<auto-comment>

Any of these fields is optional. An <instruction> has the following format:

<opname> <operand1>, . . . ,<operand>N

All but one of the comma-separated operands are optional. i.e., an operation needs at least one operand.

Space, tab, or an operator separates fields.[3] So for example it should work to say "label:sp 42", with no space between the colon and the opname, if you were so inclined. Example:

@0514:114243 a10: su a11 + 4 ; subtract memory from the AC

The following table describes each field:

Whirlwind operations – Op Codes and Operands

By default, the assembler follows the version of the instruction set described late in the Whirlwind program, in document 2M-0277 from 1958. Op-Code names are taken from that doc.

http://www.bitsavers.org/pdf/mit/whirlwind/M-series/2M-0277_Whirlwind_Programming_Manual_Oct58.pdf

Each opcode has a two- or three-letter mnemonic name, e.g. 'ca' for Clear and Add, followed by an argument. In most instructions, the argument is simply an address for an operand. There's only one addressing mode; the 11-digit field is used as an address in to Core Memory to find the operand.

A few instructions interpret the Operand field as a number rather than a memory address, and assign specific functions to bit locations, e.g., the shift instructions use operand bits to say how many bit positions the accumulator should be shifted, and what rounding mode should be used.

WWI programming depends on self-modifying code for indexed access to memory blocks, so it was common practice to use a zero in the operand field to indicate instructions that will be overwritten. Currently there's no capability to designate words in memory as "read only".

[3] Nothing depends on how many spaces or tabs are used, so tab spacing can be whatever you prefer in your fave editor (i.e., this is not python!)

By convention, memory address zero always contains zero, memory address one contains one. So "ca 1" would result in a one in the AC.

There is (currently) no (syntactic) means to declare a location as read-only.

The assembler can be configured by a directive or command-line switch to use the 1950 version of the instruction set, prior to the introduction of the 'new' I/O system, where there were a number of op-codes dedicated to specific I/O devices (i.e., the CRT display). The instruction set version is set prior to execution of the first instruction, and can't be changed during execution.

Numbers

Whirlwind programmers had a (very annoying) habit of hopping back and forth between octal and decimal numbers. It's usually possible to tell which is which, but modern coders must stay alert.

In existing Whirlwind code[4]:

- If it's a six-digit number of the form 0.00000, i.e., zero or one, a decimal point, and five numeric digits, it's an octal number
- If it's a string of digits, it's probably decimal
- If it starts with + or -, it's decimal

Given this history, in wwasm we've adopted the following formats:

- {0|1}.[0-7]^5 -- a zero or one, followed by exactly 5 octal digits, is a 16-bit unsigned octal number. For example,
  - 1.00077
  - 0.17765
- 0o[0-7]+ -- Octal, up to 6 octal digits. An Arbitrary number of leading zeroes is permitted. For example,
  - 0o40
  - 0o00070
  - 0o1777777 Out of 16-bit range
  - 0o000000000000042
- {+|-}{0|1}.[0-9]+ -- Decimal fraction, if leading digit is zero. Otherwise the "overflow" case.[Why do we need to represent this in asm?] The 16-bit word stored is the value of the fraction times 2^15, rounded to the nearest int. The number if digits accepted is arbitrary.

[4]We do actually have quite a few programs in Comprehensive System source format, although with two-character variable names and no comments, the source is not a lot easier to read than the binaries!

- -0 – A special case which evaluates to a 16-bit one's-complement negative-zero, equivalent to 0o177777.
- Bare digits -- Decimal

Wwasm flags numbers that won't fit in a 16-bit word as errors, and won't generate an output file.

zero: .word 0 ; plus zero

.word 0o137 ; gcc-style octal [LAS: 0o does not seem to be in any C std or variant]

.word 10 ; decimal integer; default bare-digits format

.word 0.00010 ; Whirlwind octal positive number;

must be five digits to the right of the fixed point

.word 1.11101 ; Whirlwind octal negative number;

must be five digits to the right of the fixed point

.word 1.77777 ; Whirlwind negative zero

.word -0 ; Whirlwind negative zero

.word +0.9 ; Whirlwind positive decimal fraction

.word +0.9999 ; Whirlwind positive decimal fraction

.word +0.99999 ; Positive decimal fraction Overflow

.word +1.0 ; Positive decimal fraction Overflow

.word -0.9 ; Whirlwind negative decimal fraction

.word -0.9999 ; Whirlwind negative decimal fraction

.word -0.99999 ; negative decimal fraction Overflow

.word -1.0 ; negative decimal fraction Overflow

Expressions

Expressions are evaluated at assembly-time to generate addresses and parameters for the program.

Evaluated expression return *values*, which may be Integer, Fraction, or Negative-zero, as described above, or String, a quoted series of any characters. Integer is the only datatype which may be manipulated by expression operations. There does not seem to be a reason to manipulate strings, fractions, or negative-zero, so those are only available as literals.

Labels and preset values (via .pp) are evaluated as variables. Parentheses may be used as needed. Operator precedence follows that of C. Operations are calculated

in integer space, as implemented in python. Ranges are checked before storing as an unsigned or one's-complement 16-bit representation or 11-bit non-negative address.

Operations supported are

- +, -: Integer addition subtraction
- Unary +, -
- *: Integer multiplication
- /: Integer division (floor (x real-div y))
- |: bit-or
- &: bit-and

Note that dot (".") and comma (",") are also designated as operators, but stay "under the covers" since they only apply to internal items such as operand lists and number text.

e.g.

.pp table_fixed_offset, 5

ca table_base + table_fixed_offset - 1

sp next_instruction

table_base:

.word 12

Labels

Wwasm accepts variable-length, relatively free-form labels. Labels may comprise any combination of letters, numbers, and underscores ('_'). Starting a label with a digit is permitted.

Note that rules for labels in Comprehensive System source code format have numerous restrictions and special cases, and require conversion to wwasm format. See M-2539-2 Comprehensive System manual and Section 2.7. None of the CS restrictions on variable names are enforced in wwasm.

Pseudo-Ops

Wwasm accepts a variety of pseudo-ops to control its behavior. None of these pseudo-ops were present in the original CS-II assembler (kind of except for the JumpTo and Program Parameter ops, which were handled as special cases).

Unless otherwise noted, *string* arguments are delimited by white-space, i.e., quotes or parens are interpreted as part of the string.

.WW_TapeID: *string*

.WW_TapeID: “fbl227p50”

This optional pseudo-op string passes along any “Tape ID” string detected by the disassembler from the 556 header into the core-file and assembler.

.WW_File *string*

.WW_File “102663328_fb131-0-2690_new_decoders_3of4.7ch/bounce_gs001_.tcore”

This optional pseudo-op string passes along the posix source file name, so that it’s visible in the output .acore file.

[Oops, looks like TapeID does have a colon and WW_File does not]

.isa [1950|1954]

.isa

The assembler defaults to the 1954 instruction set, primarily featuring the new I/O system (SI, RC and RD). Earlier single-purpose I/O commands can be used if the 1950 instruction set is chosen.

The ISA can’t be changed dynamically, i.e., the instruction set is chosen before the first instruction is executed, no matter where the .isa pseudo-op appears in the code.

.switch *name, integer*

.switch FFRegAssign, *addr1*, . . . , *addrN*

.switch LeftInterventionReg, 0.00100

.switch FFRegAssign 0o10, 27, 34, 35, 36

name must be an identifier, i.e., has the same naming constraints as a *label*. *integer* must evaluate to non-negative. For FFRegAssign the expressions must evaluate to addresses.

The .switch pseudo-op instructs the assembler to configure one of the (zillion) Whirlwind control panel switches. The *name* argument can be one of a list of named switches. The operand is a numeric value to be loaded into the switch prior to the start of simulation. Most switches are either a single bit or can contain a 16-bit word.

The .switch pseudo-op is meant to configure many of the switches, some of which are mapped into the I/O address space, accessed via si and rd instructions, some of which are hard-wired to machine behavior (e.g. *CheckAlarmSpecial*.)

The .switch settings are not dynamic; they are applied prior to the start of execution of WW code.

[The intention so far is that the switch value is configured before the simulation starts to run, i.e., it's not (as of now) interpreted as part of the instruction stream.]

Note: The assembler does not validate *name*, but just passes name and value through to the simulator.

Flip-Flop Register Preset switches are a special case; the .switch pseudo-op can preset flip-flop registers anywhere in the lower 32 words of the address space.

The simulator implements two switch registers, "LeftInterventionReg", "RightInterventionReg", and thirty two Flip-flop Register Preset Switches formatted as "FlipFlopPreset%02o".

Other switches include:

- CheckAlarmSpecial – modifies behavior of the CK ("check") instruction to either Halt or not on a mismatch. (See 2M-0277)
- FFRegAssign – modifies which FF Register appears at which address. Don't mess with this unless you know what you're doing. By default, the FF Registers are assigned addresses 0o2, 0o3, 0o12, 0o16, 0o22. [Check this!]

.dbwgt *addr|string* [, *step-value* [, *format-string*]]

.dbwgt Vxdt, 0o100

Revised Apr 2026

.dbwgt Address|ParamName [, Incr [, Format [, Min [, Max]]]]

or using keyword args, e.g.:

.dbwgt Address|ParamName, incr = 42, min = 10, max = 999, fmt = "%o"

or a combination of the two:

.dbwgt Address|ParamName, [<by-pos arg>,...,<by-pos-arg] [<keyword-arg>, <keyword-arg>]

where by-position args will be accepted in order, followed by keyword args, and a by-position arg is not permitted within the keyword-arg list.

The core file format has been extended to 5 fields, adding min and max. The code which processes these reads the new fields, but will also read the old format, so core files don't need to be reassembled.

Min and max default to 1 and 65535 respectively.

This pseudo-op adds a numerical memory address or name to the list of "debug widgets" displayed on the xwin CRT emulator. Typical use refers to a variable in the ww program. Some widgets are names, however, in which case the first operand must be a quoted string.

The simulator will display the current value of each memory location or named measure as the simulation runs, and a can be used to select and update values as the sim runs using PC Arrow keys. Up and Down arrows select and highlight the Var to be changed, Right and Left arrows increment or decrement the value. The optional *step_value* indicates what should be added or subtracted with each press of the Right or Left arrows.

The .dbwgt pseudo-ops are evaluated once by the simulator prior to execution of the first instruction.

.jumpto *addr*

.JumpTo 0o157

This pseudo-op instructs the simulator as to the starting address for the program.

The assembler doesn't pay attention to the start address, and the simulator will default to 0o40 if nothing is specified.

.org *addr*

.org 0o40

The .org pseudo-op sets the current program counter, setting the address for the next instruction word.

.daorg *addr*

.daorg

Later versions of the 556 loader format allowed specification of address in physical memory, but also the address to be used to store the file on the drum.

[I don't recall why this matters, but the tape decoder decodes it, so the Assembler passes it on. . .]

.pp *variable*, *expr*

.pp B1, 0o40000

Preset Parameter is an assembler function like a #define, to configure addresses or other values during assembly of a program. The CS-II implementation of Preset Parameters has quite a few rules on naming (see M-2539-2 Comprehensive System manual, pdf page 75) but this simulator treats preset params as additional symbols in the symbol table.

Preset Params are meant (I think) for doing address assignments and calculations, so the values associated with a PP are restricted to 0-2047. LAS I don't restrict this in the new asm

Labels used in .pp expressions must be defined before use.

E.g, this code is not legal:

.pp L201, L203 + L204

L203: ad L204

L204: .word 123

While this is legal:

L203: ad L204

L204: .word 123

.pp L201, L203 + L204

.word *expr*

.word 0o42

.word (stackend - stackbegin)/4

.word -0.31415

Insert a 16-bit word initialized with the specified value at the current address and advance the address pointer. Values may be integer, fraction, or negative-zero. Integers are converted to ww one's-complement representation if they are within the range [–(2^15 -1), +(2^15 – 1)]. Values greater than 2^15 – 1 are taken as unsigned, valid up to and including 2^16 – 1. Values outside these ranges generate an error.

.words *size* [, *initial-value*]

.words 10, 0o42

.words stackend – stackbegin + 1

.words 57, -0.31415

Insert a block of *size* 16-bit words, initialized with *initial-value* (0 if not specified) at the current address and advance the address pointer. Values may be integer, fraction, or negative-zero. Integers are converted to ww one's-complement representation if they are within the range [–(2^15 -1), +(2^15 – 1)]. Values greater than 2^15 – 1 are taken as unsigned, valid up to and including 2^16 – 1. Values outside these ranges generate an error.

.float *mantissa*:fraction, *exponent*:integer

.float +0.314159, 1

.float -0.42, 2

Calculate *mantissa*10*exponent* and convert the resulting quantity to 2 16-bit words, representing a (24,6) ww floating point number. Insert these words at the current address and advance the address pointer. *Mantissa* must be a ww fraction, and *exponent* must be an integer.

An error is generated if a give float is out of range. The maximum magnitude accepted is approximately 9.2233715E18,

A (24,6) float is supported by a library which is used by a number of other modules, including the Laning and Zierler expression interpreter. Its format is:

.flexh *string* [, *terminator*:integer]*

.*flexl *string* [, *terminator*:integer]

.flexl “abc”

.flexh “def”, 0

The .flex[h|l] pseudo-ops initialize words at the current assembly location containing flexowriter codes for the corresponding ascii characters in *string*. .flexl packs each code into the low end of the word, and .flexh packs each code into the high end (top six bits) of the word.

The second operand is an optional integer terminator word added to the sequence of code words.

Note that there are many ASCII characters that don’t have Flexo equivalents (e.g. “[“ or “;”), and if specified will raise an error.

.exec *python-stmt*

.exec ring-bell (loudness=42)

Bypass the usual simulator flow and execute a python statement. Note this is the only wwasm pseudo-op or ww-op which does not do the standard operand evaluation. The text following .exec is taken verbatim to the simulator.

This pseudo-op is executed *before* the following instruction, i.e.:

```
ca 12
cp branch_taken
.exec if debug: print branch_not_taken
cp 13

branch_taken:
ad 14
```

.print *format-str*, *addr1*, . . . ,*addrN*

.print “X=%d, Y=%d”, X, Y

addr-i is an expression which must evaluate to an address. *format-str* must be a quoted string.

Formats

- %d, %o – format the corresponding memory addresses as decimal or octal
- %ad, %ao – insert the content of the accumulator, either in decimal or octal

This pseudo-op bypasses the usual simulation flow and causes Python to print a message, typically for debug. The string can contain printf-like formatting indicators, followed by a corresponding list of memory addresses.

The .print statement is executed prior to the following instruction, as with .exec.

.include *filename*

.include “float-lib.ww”

Assemble the current file including the text from *filename,* effectively inserting the text into the file being assembled. The result is an assembly of the two files as though they were one. The included file must be a syntactically valid .ww file.

The listing denotes the inclusion with markers in comments:

; .include "float-lib.ww"

. . . [float-lib.ww statements]. . .

; end .include "float-lib.ww"

Multiple includes may appear in a file, and includes may be nested to any level. Circular includes result in an error. [I think]

The –Reformat option to the assembler will not process includes; i.e., it just reformats the given top-level ww file.

.if *<expr>*

Use .if as a prefix, followed by a literal integer or variable (settable via .pp), then any instruction. For example:

.pp Debug, 1

.if Debug .print "x=%o", x

...

.if 0 ca y

...

x: .word 42

y: .word 57

...

So the .if appears as a prefix, and nothing else can precede or follow it except an op or pseudo-op. I.e., no label or prefix-addr can appear with a .if.

In the above example, Debug allows one to control whether ,print or .exec or anything gets assembled. .if 0 is like commenting-out code, but you can assign a pp to it to include a sequence of instructions or not.

.if causes statements to appear in the core and listing, or not. The .if itself will never appear in a listing, with the possible exception of the --Reformat option, but I'll have to check the difficulty of that. The listing will still assemble, no change there.

.simparam

Assembler source can include ".simparam" pseudo-ops to direct the simulator to apply certain configuration parameters when it starts. The assembler doesn't interpret the .simparam lines, it just passes them through to the simulator via the .core file.

.simparam Radar = 1, CrtFadeDelay = 8, NoAlarmStop = 1, AutoClick = 1

Tags that the simulator accepts (as of Jul 2026) are

"isa", "Radar", "CrtFadeDelay", "NoAlarmStop", "AutoClick", OffsetCrtX, OffsetCrtY

[OffsetCrt[x|y] only appear as sim params and instruct the graphic module to shift the image on the screen... e.g., Blackjack is squeezed right up against the top

Future .simparams

These are ones that I tinker with while debugging tapes, are not normally needed, but once in a while a program requires, or at least, always uses the arg. So I'd keep the cmd line but also add a simparam

--FlexoWin Display Flexowriter output in its own window

--PETRAfile PETRAFILE

File name for photoelectric paper tape reader A input file

--PETRBfile PETRBFILE

File name for photoelectric paper tape reader B input file

--ZeroizeCore Return zero for uninitialized core memory

These seem like they should only appear in the core file.

--NoToggleSwitchWarning

Suppress warning if WW code writes a read-only toggle switch

--NoZeroOneTSR Don't automatically return 0 and 1 for locations 0 and 1

Listing and Source File

The wwasm assembler is intended to fit in a tool chain mostly optimized around reverse-engineering archival Whirlwind material. As such, a critical use of the assembler is to re-assemble disassembled code as labels and comments are added, and to accommodate modest changes to program flow when necessary. To accomplish this goal, the assembler is designed to read its own output listing.

A typical flow might be:

1) Run a binary object (.tcore file) in the simulator to start to understand the flow
2) Disassemble the .tcore to obtain an initial uncommented .ww source file
3) Start adding comments and labels to the .ww file
4) Assemble the .ww file to produce a .lst file, showing source code plus comments, along with the octal representation of each instruction and variable in memory.
5) Re-name the .lst file with a .ww extension.
6) Run the simulation. Labels from the source will show up in simulation traces.
7) Keep editing the new .ww to add more comments and labels, plus debug .print statements as needed
8) Until (Done or It's_Dinner_Time), GoTo (f).

CS-II Code Converter

[this currently is a separate program that converts a file in CS-II format into .ww format]

Core File Format

The .ww files give source code in a more-modern assembler format than the WW guys used, so I hope that's more readable than CS-II source. The assembler outputs a listing .lst file (which can be used again as input to the assembler!) and a .acore file, which contains the core image.

The core image is ascii, and essentially describes what values to load into memory prior to launching the program execution. Any locations not specified, or indicated in the core file as None are set to None in the simulator's memory, triggering an error if the location is used before being initialized.

Lines in the file have the following formats:

- @Cnnnn: gives eight 16-bit octal words, starting at octal address nnnn. Use none to indicate that a location is uninitialized. Values to be loaded are always octal, up to seven digits long.
- @Snnnn: gives one entry in the symbol table. 'nnnn' is the address of the symbol, the name of the symbol follows the colon..
- @Nnnnn: records the source file comments (if any) for a given address word.
- @Ennnn gives lines that contain pseudo-ops.
- %JumpTo nnn gives the start address
- %TapeID: optionally gives the ASCII string used as the identifier for this binary
- %File: optionally gives the source file name

Lines which begin with a semicolon are treated as core-file comments and ignored. Blank lines are ignored.

In general, a ".tcore" represents the image read from a tape and reformatted from 5-5-6 into memory locations. A ".acore" file is generally the output of the assembler, which may have symbols and comments embedded. The two files have identical structure, just different sources.

For example:

; *** Core Image ***

%File: 102766750_fb131-0-2692_number_display.7ch

%TapeID: fbl3l-0-2692

%JumpTo 0o40

@C0000: 0000000 0000001 0000000 0000000 None None None None

@C0040: 0100200 0040002 0100201 0040003 0100202 0040600 0100203 0040601

@S0276: dot_xpos

@S0277: acctmp

@S0300: do_dot

@S0500: u0500

@S0501: chr_cnt

@N0040: load constant 0o12 into AC

@N0041: Store it in a FF Reg(??)

@N0042: load constant 0o140510 into AC

@E0042: print: "execute .print" ["1", "2"]

@E0043: exec: if True: print("executed .exec")

Assembler Grammar

The tokenizer and parser in wwasmparser.py follow this grammar fairly well, but there are probably a few spots where it doesn't line up. The code version wins unless it's shown to be some kind of error. Something not captured well in this grammar is some special casing needed for .exec.

Note that the grammar tries to capture the nuances of number denotation, but in the code we use dot (".") as an operator and distinguish the number types in eval. This frees us from having to add lexical knowledge of those number forms.

Informal grammar of a single line, showing what's optional:

<line> = [@ <oct> [. <dec>] : <oct> <sp>] [<label> : <sp>]

[<op> <sp> <operand>[, ..., <operand>] [<sp>]] [<comment>] [<auto-comment>]

More formal version, with optionality omitted. Since so much is optional, we'll assume that's handled in what follows, to simplify the presentation.

<line> = <prefix-addr> <sep> <label> <sep> <instruction> <sep> <comment>

<sep> <auto-comment>

Clauses for the components:

<prefix-addr> = <oct> . <dec> : <oct>

<label> = <identifier> :

<instruction> = <op> <whitespace>+ <expr>

<op> = <ww-op>|<pseudo-op>

<ww-op> = ta|ao|...

<pseudo-op> = .word|.org|...

<operand> = <comma-expr>

<expr> = <comma-expr>

<oct> = <digit-seq> Digits 0-7 allowed

<dec> = <digit-seq> Digits 0-9 allowed

<number> = <oct-num> | <dec-num>

<oct-num> = 0o<oct> | {0|1}.{0-7}^5

<dec-num> = {+|-}{0|1}.<dec> | <dec>

Tokens (lexical level):

<sep> = <separator> = <whitespace>+ | <operator>

<operator> = + | - | * | / | @ | @@ | : | . | ; | , | 0o | & | |

<identifier> = <extended-alphanumeric-char>+ Can't be all digits

<digit-seq> = {0-9}+

<comment> = ; <any-char-but-not-@@>*

<auto-comment> = @@ <any-char>*

<extended-alpha-char> = <alpha> | _

<extended-alphanumeric-char> = <extended-alpha-char> | <numeric-char>

<string> = " <any-char>* "

Expressions:

<comma-expr> = <additive-expr> | <additive-expr> , <comma-expr>

<additive-expr> = <multiplicative-expr> | <multiplicative-expr> {+ | -}

<additive-expr>

<multiplicative-expr> = <bit-or-expr> | <bit-or-expr> {* | /} <multiplicative-expr>

<bit-or-expr> = <bit-and-expr> | <bit-and-expr> '|' <bit-or-expr>

<bit-and-expr> = <dotted-expr> | <dotted-expr> & <bit-and-expr>

<dotted-expr> = <unary-expr> | <unary-expr> . <dotted-expr>

<unary-expr> = <atomic-expr> | {+ | - | 0o} <unary-expr>

<atomic-expr> = <identifier> | <digit-seq> | <string> | (<expr>)

Simulator

The MitWE simulator offers instruction-level emulation of the Whirlwind instruction set, as defined (mostly) in 2M-0277, including a number of I/O devices (but not all). It does not emulate the internal timing "micro states" within a single instruction.

The simulator operates by first reading a "core file" into its simulated 2K memory unit. Upon start, it goes the address of the first instruction, fetches it, executes it, goes to the next, etc., until it hits an alarm or halt, or the cycle count specified on the command line runs out.

Command Line Options

There are quite a few command line arguments to the simulator, although the only one that's actually needed is the name of the core file containing code to be run. In many cases, *none* of the additional arguments may be required.

usage: Run a Whirlwind Simulation.

[-h] [--LogDir LOGDIR] [--ArchaeoLog] [-t] [-a] [-f]

[-fo FLOWGRAPHOUTFILE] [-fd FLOWGRAPHOUTDIR]

[-j JUMPTO] [-q] [-v] [--NoWarnings] [-D] [-c CYCLELIMIT]

[--CycleDelayTime CYCLEDELAYTIME] [--AutoClick]

[--AnalogScope] [--xWinSize XWINSIZE] [--FlexoWin] [--NoXWin]

[--NoToggleSwitchWarning] [--LongTraceFormat]

[--TraceCoreLocation TRACECORELOCATION]

[--PETRAfile PETRAFILE] [--PETRBfile PETRBFILE] [--NoAlarmStop]

[--QuickStart] [-n] [-p] [-b] [-m] [--NoZeroOneTSR]

[--SynchronousVideo] [--CrtFadeDelay CRTFADEDELAY]

[--DumpCoreToFile DUMPCORETOFILE] [--RestoreCoreFromFile RESTORECOREFROMFILE]

[--DrumStateFile DRUMSTATEFILE] [--MuseumMode] [--HnfProgramDispatcher]

[-d] [--ZeroizeCore] [-map]

[--TTYname TTYNAME] [--LEDbrightness LEDBRIGHTNESS]

[--CRTgeometry CRTGEOMETRY]

corefile

positional arguments:

corefile file name of simulation core file

options:

-h, --help show this help message and exit

--LogDir LOGDIR Directory into which to store logs. Default is current wd.

--ArchaeoLog Write data to the archaeolog dir.

-t, --TracePC Trace PC for each instruction

-a, --TraceALU Trace ALU for each instruction

-f, --FlowGraph Collect data to make a flow graph. Default output file <corefile-base-name>.flow.gv

-fo FLOWGRAPHOUTFILE, --FlowGraphOutFile FLOWGRAPHOUTFILE

Specify flow graph output file. Implies -f

-fd FLOWGRAPHOUTDIR, --FlowGraphOutDir FLOWGRAPHOUTDIR

Specify flow graph output directory. Implies -f

-j JUMPTO, --JumpTo JUMPTO

Sim Start Address in octal

-q, --Quiet Suppress run-time messages (nop -- here just for compat)

-v, --Verbose Produce run-time messages

--NoWarnings Suppress Warning messages

-D, --DecimalAddresses

Display trace information in decimal (default is octal)

-c CYCLELIMIT, --CycleLimit CYCLELIMIT

Specify how many instructions to run (zero->'forever')

--CycleDelayTime CYCLEDELAYTIME

Specify how many msec delay to insert after each instruction

--AutoClick Execute pre-programmed mouse clicks during simulation

--AnalogScope Display graphical output on an analog CRT

--xWinSize XWINSIZE specify the size of an xWinCrt pseudo-scope display in pixels

--FlexoWin Display Flexowriter output in its own window

--NoXWin Don't open any x-windows

--NoToggleSwitchWarning

Suppress warning if WW code writes a read-only toggle switch

--LongTraceFormat print all the cpu registers in TracePC

--TraceCoreLocation TRACECORELOCATION

Trace references to Core Memory Location <n> octal

--PETRAfile PETRAFILE

File name for photoelectric paper tape reader A input file

--PETRBfile PETRBFILE

File name for photoelectric paper tape reader B input file

--NoAlarmStop Don't stop on alarms

--QuickStart Don't wait for the Restart button on the control panel; just go!

-n, --NoCloseOnStop Don't close the display on halt

-p, --Panel Pop up a Whirlwind Manual Intervention Panel window

-b, --BlinkenLights Activate a physical Whirlwind Manual Intervention Panel

-m, --MicroWhirlwind Activate the MicroWhirlwind Model

--NoZeroOneTSR Don't automatically return 0 and 1 for locations 0 and 1

--SynchronousVideo Display pixels immediately; Disable video caching buffer

--CrtFadeDelay CRTFADEDELAY

Configure Phosphor fade delay (default=0)

--DumpCoreToFile DUMPCORETOFILE

Dump the contents of core into the named file at end of run

--RestoreCoreFromFile RESTORECOREFROMFILE

Restore contents of memory from a core dump file

--DrumStateFile DRUMSTATEFILE

File to store Persistent state for WW Drum

--MuseumMode Cycle through states endlessly for museum display

--HnfProgramDispatcher

Activate the special-purpose Demo Program Dispatcher for use at HNF

-d, --Debugger Start simulation under the debugger

--ZeroizeCore Return zero for uninitialized core memory

-map, --MemoryMap Produce a memory map (.map) file of the access types during this run

--TTYname TTYNAME Configure TTY name to access external media controller

--LEDbrightness LEDBRIGHTNESS

Configure MicroWhirlwind LED intensity as three ints; r, w, b

--CRTgeometry CRTGEOMETRY

Configure x-window screen geometry as <width>x<height>{+-}<xoffset>{+-}<yoffset>

Program Launch Options

This group of options sets up the behavior of the simulation before the first instruction is fetched.

-j JUMPTO, --JumpTo JUMPTO

Sim Start Address in octal

JumpTo gives the start address for the first instruction. This defaults to 0o40 if not specified on the command line or in the core file. If both are present, the command line has the final say on where to start.

-D, --DecimalAddresses

Display trace information in decimal (default is

octal)

WW programmers switched back and forth between Octal and Decimal with dismaying alacrity. Sometimes it's easier to debug when the traces are in the same base that the programmer was using.

-c CYCLELIMIT, --CycleLimit CYCLELIMIT

Specify how many instructions to run (zero->'forever')

Many Whirlwind programs were written to "run forever"; this option allows a way to run some cycles and then bail out. The default, or a cycle limit of zero, causes the simulator to run 'forever' until a halt or some manual intervention.

--CycleDelayTime CYCLEDELAYTIME

Specify how many msec delay to insert after each

Instruction

CycleDelayTime intentionally slows the simulation to "human speed". Sometimes useful to see what order graphics are being displayed.

--AutoClick Execute pre-programmed mouse clicks during simulation

Some simulations invoke a mechanism on startup to 'automatically' click a bunch of buttons or switches before reverting to user input for light-gun hits. Examples are Air Defense and Nim, both of which have startup sequences that need to be repeatable for reliable debug. The Autoclick function is mostly implemented in per-application project_exec.py files.

-r, --Radar Incorporate Radar Data Source

This is a special-purpose argument that turns on the simulated radar system, specifically for the air-defense M-1343 simulation. Turning this on does have the (intended) side effect of repurposing two Toggle Switch Register addresses.

--NoAlarmStop Don't stop on alarms

Default WW behavior is to halt on any alarm (especially annoying for arithmetic overflow. Often it's easier to debug a program by ignoring the overflows and following the flow, then going back later to see what caused the overlow.

--NoZeroOneTSR Don't automatically return 0 and 1 for locations 0 and 1

The convention for Whirlwind programmers was to put constant zero in location zero, and constant one in location one. The simulator does that by default. But it is possible to change those first two locations (e.g., they could be used as the first two words of a program entirely in TSR).

--PETRAfile PETRAFILE File name for photoelectric paper tape reader A input file

--PETRBfile PETRBFILE File name for photoelectric paper tape reader B input File

There are I/O instructions for reading from the two Photo Electric Tape Readers. If encountered, the simulator will read bytes from the specified file to represent whatever would have been on the tape.

--DrumStateFile DRUMSTATEFILE

File to store Persistent state for WW Drum

There are instructions to read and write the drum; file is used as the non-volatile backing store for the emulated drum device

--DumpCoreToFile DUMPCORETOFILE

Dump the contents of core into the named file at end

of run

--RestoreCoreFromFile RESTORECOREFROMFILE

Restore contents of memory from a core dump file

These two optional args make it possible to glue two simulation runs together by carrying core memory state forward from one to the next.

Display Options

A critical part of the WW design was the Oscilloscope Display. i.e., a device for drawing points and vectors on a large cathode-ray-tube. Coupled with that was the Light Gun[5] pointing device.

The Oscilloscope Display was not used in every program. In the WW simulator, by default, the Oscilloscope is emulated in a graphical X-Window created specifically for that purpose. The graphics window is only created on the first access to the display device by the WW program, so it's not instantiated at

[5]Fear Not! This 'gun' shoots nothing! However, it does *detect* photons from the CRT display. Even the TSA agrees that this 'gun' can be carried on an airplane.

all unless required. Also by default, mouse clicks in the graphics window are interpreted by the simulator as "light gun hits". The 2M-0277 documents which objects can be seen by the light gun and which cannot;[6] the simulator does the same as best we can.

There are a few arguments that control the behavior:

--AnalogScope Display graphical output on an analog CRT

This complex argument switches the display from an emulated oscilloscope on the laptop to a real analog oscilloscope, driven in X/Y/Z mode the same way as the original WW displays. This option only works on RaspPi with the Rainer display adapter (which needs an xref).

-p, --Panel Pop up a Whirlwind Manual Intervention Panel

The Panel option activates an additional window emulating the Whirlwind 'control panel.' (Yeah, ok, it was a Control Room with hundreds of buttons and lights. This is a small subset of the key control points.) See notes in 3.2.

--NoXWin Don't open any x-windows

NoXWin prevents a graphics window from being popped up at all.

-n, --NoCloseOnStop Don't close the display on halt

Normally when the WW simulator hits a halt or a trap, the sim process terminates and the oscilloscope display is closed. NoCloseOnStop holds the window in suspended animation until the simulator process is terminated. This is handy for debugging when the code hits an unexpected trap while painting the screen.

--SynchronousVideo Display pixels immediately; Disable video caching buffer

In the Real Whirlwind™, each graphics instruction has an immediate result on the CRT. There's no buffering, no rasters, just voltages and points. With a modern PC-style display, that's pretty inefficient, so the default behavior is to buffer a series of screen updates. SynchronousVideo causes the simulator to draw each and every point one at a time. Helpful for debug when you want to see which part of the code is drawing what part of a display.

--CrtFadeDelay CRTFADEDELAY

Configure Phosphor fade delay (default=0)

The CRT emulator must not only draw points and lines, but erase them after a short delay to yield the effect of phosphor decay. This knob allows some tuning of the phosphor decay characteristics. The default almost always works, without using this parameter.

--LEDbrightness LEDBRIGHTNESS

[6] Basically, the light gun will respond to Points but not Lines or Characters on the WW display

Configure MicroWhirlwind LED intensity as three ints; r,w,b

Using the argument --LEDbrightness 0 prints the default LED brightness values

--CRTgeometry CRTGEOMETRY

Configure x-window screen geometry as <width>x<height>{+-}<xoffset>{+-}<yoffset>

The geometry argument gives the size and position on the screen. For a dual-CRT, an increasing x offset will shift the windows from the left screen onto the right screen.

Instruction Trace

This group of arguments controls the type of trace information generated during a simulation run. The default is to print a line of output for each branch and each I/O instruction. [I'm thinking of changing the default to "-q" to cut the noise!]

-q, --Quiet Suppress run-time message

Quiet suppresses most of the non-error messages, so there's less clutter on the console

-t, --TracePC Trace PC for each instruction

--LongTraceFormat print all the cpu registers in TracePC

TracePC outputs one line for each instruction executed, showing PC, instruction and a few of the key registers, allowing the Reverse Engineer to see where the program is going. Capturing this output in a file yields a way to retrospectively see "how the heck did we get here?". LongTraceFormat adds a couple more registers, but it's rarely needed (by me, anyways!)

--TraceCoreLocation TRACECORELOCATION

Trace references to Core Memory Location <n> octal

TraceCoreLocation prints a line each time a specific core location is read or written

-a, --TraceALU Trace ALU for each instruction

TraceALU turns on more detail on each arithmetic operation. Essential for figuring out if Ones Complement is working, Too Much Information for anything else.

-f, --FlowGraph Collect data to make a flow graph

See section 3.5. This complicated feature generates a 'flow chart' for the program as executed

--NoToggleSwitchWarning

Suppress warning if WW code writes a read-only toggle

Switch

The Toggle Switch Registers (TSR) are not writable (they're supposed to be physical switches, dude!) But that doesn't mean the first 32 words are all unwritable. The five Flip Flop registers overlay TSR address space, and they are writable. In the real machine, and by default in the simulator, writes to TSR are simply ignored, as they were often used when the programmer needed the side effect of a write instruction, but didn't want the result. This flag issues a warning if a write to unwritable TSRs is executed.

Specialized

These are meant specifically for unattended museum exhibit use.

--MuseumMode Cycle through states endlessly for museum display

--MidnightRestart Restart simulation daily at midnight

Control Panel

[New, as of Apr 2024]

The –Panel option for the simulator pops up a "Control Panel" similar to what the machine operator would have seen in the Whirlwind control room. [I need a Blog Article on this; xref the CHM object], allowing the sim user to set parameters and run programs more-or-less as they did on the Real Machine™.

There's a Graphical Code to follow:

- Orange is an Indicator light
  - Orange circle with a black center is a Zero indication.
  - A filled orange circle shows a One indication.
  - Clicking an indicator has no effect.
- White is a switch or push button
  - White outline with a black center is a switch set to Zero
  - White outline with a blue center is a switch set to One.
  - Clicking on a switch will change its setting.

All indicators are updated as the simulator runs.

The top segment of the panel represents the two "Manual Intervention Registers" with corresponding Activate buttons. Each MIR ("Left" and "Right") represents one 16-bit number, coded as five one-of-eight radio buttons (and a "one of two" radio button for the extra bit!).

The two Activate buttons are ordinary push buttons; any press "sets" the corresponding Activate bit and turns on the lamp; the lamp is cleared when the bit is read by WW software. (For some reason, they're "Upper" and "Lower" Activate buttons; I'm assuming Upper is on the left).

The next rows represent a subset of the five possible Flip Flop Registers. Each one displays its current state in 16 indicators. The associated row of buttons (really Toggle Switches) gives the preset values. When WW starts, or executes the Reset FF instruction, the value in the toggle switches is copied into the corresponding register.

While in the "real machine" toggle switches would have to be set manually, the simulated switches are initialized to whatever the FF Register Preset values optionally in the core file. [check the name]

The simulator currently shows four of the five FF Registers, indexed by their conventional address in the lower 32 address locations. In practice, FF2 and FF3 are often used for parameter input, while the others are not.

The next block of indicators shows the state of various registers, including Accumulator, B Register, AR and Program Counter. The row of switches gives a preset value for the Program Counter, used by Start Over (below). By default, the simulator sets the PC Preset switches to the Jump-To address if there is one in the core file.

The bottom row controls the run-time operation of the machine.

In normal operation without the control panel, the simulator wakes up in Run state, having already "read in" the core file, and immediately fetches the first instruction. However, with –Panel, the machine wakes up in Stop state and waits for a button to start.

- **Start Over** will reset the PC to whatever is in the PC Reset switches (defaulting to the Jump-To address).
- **Restart** will simply start to execute at whatever address is in the PC
- **Start at 40** will reset the PC to 0o40 (the default start address for many WW programs) and run from there.
- **Stop** sets the run state to stop, leaving registers all the way they were when the Stop button was pressed. Restart will pick up from wherever it left off.
- **Order-by-Order** is like Restart, but it runs just one instruction and then returns to Stop state.
- **Readin** starts over by reading in a new core file.

Other settings don't change the Run state directly:

The Examine button only works when the machine is Stopped, and reads the memory location corresponding to the PC Preset register, leaving the result in

the PAR/MDR[7] indicators.[8]

I/O Devices

By the end of the project, Whirlwind had a lot of I/O gear attached. The simulator includes some of it:

- Flexowriter
- Teletype (rarely used)
- Graphics CRT & Light Gun
- Photo Electric Paper Tape reader(s)
- Drum Storage

In addition, there are special-purpose I/O hacks for radar and light-gun to work with the M-1343 Air Defense demonstration.

Graphics Output and the Light Gun

Whirlwind had large CRTs used for graphical output, and (through the light gun) input. The programming and hardware interface to these devices is *much* simpler than conventional raster graphics devices, simply using a pair of D/A converters to position a beam on an x-y oscilloscope display, with a single bit for each 'scope to turn the beam on and off. As a result, the WW programmer is responsible for repainting the display often enough to keep the flicker down to an acceptable level. Real-time programming indeed.

Graphics primitives can draw points and line segments, with a hardware feature added later to trace out a seven-segment character as a series of short segments.

[picture]

For a modern programmer, this programming model is completely counter-intuitive. There are commands to draw points and lines, but there are no commands to erase them or clear the screen. That's because the WW instructions draw the point on CRT phosphor in about 65 usec, lighting it up for a fraction of a second before the natural decay of the phosphor causes the point to be extinguished. If you want a stable display, you need to keep redrawing it often enough to hide the inherent flicker.

The simulator has two [for now] ways of emulating this I/O mechanism.

- On RasPi, Windows or Mac, the simulator calls a library that creates a window on the screen, and emulates the drawing of points and lines on

[7] In the WW docs, this indicator seems have been added to debug the data going into Parity Check circuitry, but it always shows the contents of the last memory location read, i.e., it's a Memory Data Register, so it's unclear if it's a PAR or MDR. The last address used is shown in the Memory Address Register indicator, which is unambiguously a MAR.

[8] You can Peek, but you can't Poke

'phosphor' that automatically fades with time. Mouse clicks in the graphics window can be interpreted as Light Gun hits.

- On RasPi only, the simulator alternately can activate Rainer Glaschick's "WWI-VecIF" hardware interface, which drives one or more analog oscilloscopes, using circuitry much closer to the "real" Whirlwind. This interface also includes a physical Light Gun that senses points drawn on the CRT in the same way the real WW did.

Simulator Files

Core Files

See Section 2.11.

Project_exec.py Files

Program Trace Log

Flow Graph

Ancillary Tools

wwdisasm – Disassembler for Whirlwind Binaries

wwdiff – Compare core files

wwutd – Universal Tape Decoder

End of Document